

Fundamentals of program development

Arithmetics with integers and floating point numbers

Chalmers

Agenda

Representations of numbers

Integers and floating point numbers

Rounding and other conversions

Algorithms manipulating numbers

Integers

Binary = Base 2 numeral system

1	0	1	1
$1 \cdot 2^3$	$+ 0 \cdot 2^2$	$+ 1 \cdot 2^1$	$+ 1 \cdot 2^0$
$1 \cdot 8$	$+ 0 \cdot 4$	$+ 1 \cdot 2$	$+ 1 \cdot 1$
8	+ 0	+ 2	+ 1
11			

Every position corresponds to a power of 2.

1	7	9	3
$1 \cdot 10^3$	$+ 7 \cdot 10^2$	$+ 9 \cdot 10^1$	$+ 3 \cdot 10^0$
$1 \cdot 1000$	$+ 7 \cdot 100$	$+ 9 \cdot 10$	$+ 3 \cdot 1$
1000	+ 700	+ 90	+ 3
1793			

Compare with decimal numbers:
Every position corresponds to a power of 10.

Binary = Base 2 numeral system

1	0	1	1
----------	----------	----------	----------

$$1*2^3 + 0*2^2 + 1*2^1 + 1*2^0$$

$$1*8 + 0*4 + 1*2 + 1*1$$

$$8 + 0 + 2 + 1$$

$$= 11$$

Every position corresponds to a power of 2.

1	7	9	3
----------	----------	----------	----------

$$1*10^3 + 7*10^2 + 9*10^1 + 3*10^0$$

$$1*1000 + 7*100 + 9*10 + 3*1$$

$$1000 + 700 + 90 + 3$$

$$= 1793$$

Compare with decimal numbers:
Every position corresponds to a power of 10.

decimal	binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001

decimal	binary
10	1010
11	1011
12	1010
13	1101
14	1110
15	1111
16	10000
17	10001
18	10010
19	10011

decimal	binary
20	10100
21	10101
22	10110
23	10111
24	11000
25	11001
26	11010
27	11011
28	11100
29	11101

Native Integers

- Computer hardware can perform arithmetic on integers which fit in one machine word.
- If the word size is 64, there is space for $2^{64} = 18446744073709551616$ integers in a word.
 - Arithmetic is modulo 2^{64} . That is, x and $x+2^{64}$ are the same.
 - The machine can then handle integers in the interval $[-2^{63}, 2^{63}-1]$, or $[-9223372036854775808, 9223372036854775807]$
 - Rule of thumb: 20 decimal digits won't fit

32 bits are not enough for everything

Unix time

文A 34 languages ▾

Unix time is the number of seconds that have passed since the initial **date** and **time** known by UNIX as Tuesday 1 January 1970 00:00:00 UTC.^[2] It is mostly used in **Unix-like** and many other **operating systems**.

People can check Unix time on most Unix computers by typing `date +%S` on the **Unix command line**.

On 19 January 2038, Unix time will be 2 147 483 647, or $2^{31}-1$.

32-bit numbers are not enough!

Current Unix time

1757454307 (update)

(2025-09-09T21:45:07+00:00)



Unix time passed 1 000 000 000 seconds in 9 September 2001 01:46:40 UTC.^[1]

64 bits are not enough for everything

To create really big numbers, you can use factorials:

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

The implementation comes soon

If we had 64-bit machine integers (like in Python 2) this would have happened!

But Python 3 does not expose machine integers...

```
1
2
6
24
120
720
5040
40320
362880
3628800
39916800
479001600
6227020800
87178291200
1307674368000
20922789888000
355687428096000
6402373705728000
121645100408832000
2432902008176640000
-4249290049419214848
-1250660718674968576
8128291617894825984
-7835185981329244160
```

The integers in Python 3 are represented by a complex data structure, similar to a list of machine-level integers.

Therefore, it is only the size of the working memory which limits their size.

They are much more costly to manipulate than machine-level integers. If efficiency is a concern, you can use a library like *numpy* that uses machine-level integers.

```
1
2
6
24
120
720
5040
40320
362880
3628800
39916800
479001600
6227020800
87178291200
1307674368000
20922789888000
355687428096000
6402373705728000
121645100408832000
2432902008176640000
51090942171709440000
1124000727777607680000
25852016738884976640000
620448401733239439360000
```

Factorial

```
def factorial(n):  
    fac = 1  
    for k in range(2, n+1):  
        fac = fac * k  
    return fac
```

This computes:

$$n! = 2 \cdot 3 \cdot \dots \cdot n$$

Example: $n = 6$

	fac = 1
k = 2:	fac = 1 * 2 → 2
k = 3:	fac = 2 * 3 → 6
k = 4:	fac = 6 * 4 → 24
k = 5:	fac = 24 * 5 → 120
k = 6:	fac = 120 * 6 → 720
	return 720

Floating point numbers

Literal floating point numbers in Python

Using a decimal point and not a comma:

`3.14159`

You can also start with a dot if the integer part is zero:

`.25 == 0.25`

The “scientific” notation: `<mantissa>e<exponent>`

`22.5e6 == 22500000.0` and `2.25e-6 == 0.00000225 == 22.5e-7`

The same as $22.5 * 10^6$ and $2.25 * 10^{-6}$

Fractional Binary

1	.	0	1	1
----------	----------	----------	----------	----------

$$1*2^0 + 0*2^{-1} + 1*2^{-2} + 1*2^{-3}$$

$$1 + 0/2 + 1/4 + 1/8$$

$$1 + 0 + 0.25 + 0.125$$

$$= 1.375$$

Every position corresponds to a power of 2.

1	.	7	9	3
----------	----------	----------	----------	----------

$$1*10^0 + 7*10^{-1} + 9*10^{-2} + 3*10^{-3}$$

$$1 + 7/10 + 9/100 + 3/1000$$

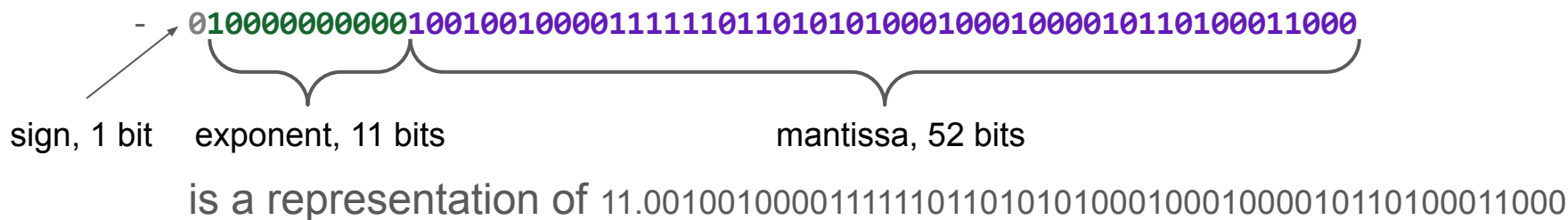
$$1 + 0.7 + 0.09 + 0.003$$

$$= 1.793$$

Compare with decimal numbers:
Every position corresponds to a power of 10.

Floating point (IEEE standard)

- A floating point number $-1^s \cdot m \cdot 2^{e+b}$ is represented by
 - a triple s, m, e (sign, mantissa, exponent) of integers
 - packed in a single machine word
- Example for a 64-bit machine:



(The first bit of the mantissa isn't stored because it is always 1; b is a bias, helps e.g. represent negative exponents)

- The IEEE standard is widely used, across various computer architectures and programming languages

Floating point is not the same as real number

- Implementation of irrational numbers as floats are only approximations

```
>>> import math
>>> math.sin(math.pi)
1.2246467991473532e-16
```

- Even, most rationals are approximated

```
>>> s = 0
>>> for i in range(1000):
...     s = s+(1/1000)
...
>>> s
1.0000000000000007
```

Testing if a floating point variable x is equal to a given value v is almost always wrong!

Use:

```
abs(x - v) < epsilon
```

or

```
math.isclose(x, v)
```

Predefined constants

```
>>> import math
```

```
>>> math.pi
```

```
3.141592653589793
```

```
>>> math.e
```

```
2.718281828459045
```

The value of π ?

The value of [Euler's number](#)?

No, these can only be **approximations**.

Conversion from int to float can lead to precision loss

```
>>> factorial(100)
93326215443944152681699238856266700490715968264
38162146859296389521759999322991560894146397615
65182862536979208272237582511852109168640000000
0000000000000000
```

```
>>> float(factorial(100))
9.332621544394415e+157
```

```
>>> int(float(factorial(100)))
93326215443944150965646704795953882578400970373
18409883101288954058222723857043129506611308928
83272778258496640065242705545359762897193828521
81865895959724032
```

integers are exact

floating point numbers are approximations

Conversion back does not give the correct result because the above number requires more than 53 bits of precision in the mantissa.

Operators and implicit conversions

```
>>> 1 + 0.5
```

```
1.5
```

```
>>> 0.5 * 7
```

```
3.5
```

```
>>> 7 // 2
```

```
3
```

```
>>> -7 // 2
```

```
-4
```

```
>>> 7 / 2
```

```
3.5
```

In Python, operators automatically promote integers to floating point numbers. *Usually* what you want.

Division is a bit tricky:

`x // y` is integer division

(rounded towards $-\infty$)

`x / y` is floating point division

Factorial with floating point numbers

```
def factorial(n):  
    prod = 1.0  
    for k in range(2,n+1):  
        prod = prod * k  
    return prod
```

This computes:

$$2.0 \cdot 3.0 \cdot \dots \cdot n$$

Note that it is much faster to compute with floating point values than with very large integers.

```
1.0  
2.0  
6.0  
24.0  
120.0  
720.0  
5040.0  
40320.0  
362880.0  
3628800.0  
39916800.0  
479001600.0  
6227020800.0  
...  
1.2296942187394488e+289  
2.0044015765453015e+291  
3.2872185855342945e+293  
5.423910666131586e+295  
9.003691705778433e+297  
1.5036165148649983e+300  
2.526075744973197e+302  
4.2690680090047027e+304  
7.257415615307994e+306  
inf  
inf
```

Special floating point values

- infinities: `inf` and `-inf`
 - when you exceed the maximum possible value
 - libraries (numpy...) will give you infinities rather than throwing exceptions if possible.
- "Not a number": `nan`.
 - For example the result of `+0*inf` or `inf/inf`.

Typically, `inf` and `nan` indicate an error in a program. But they can also be legitimate results for certain input data points.

Note also that `+0` and `-0` are different floating point values. Which one you have depends on the sign bit.

Exponentiation and absolute value

```
>>> 2 ** 3
```

```
8
```

** raises to a power

```
>>> 2 ** 0.5
```

```
1.4142135623730951
```

power 0.5 is the square root

```
>>> 2 ** -1
```

```
0.5
```

negative powers create reciprocals

```
>>> abs(123)
```

```
123
```

abs() is the **absolute value**, $|x|$ in usual math notation

```
>>> abs(-123)
```

```
123
```

always positive or 0

Remainder

```
>>> 10 % 3
```

```
1
```

```
>>> n = int(input())
```

```
5
```

```
>>> if n % 2 == 0:
```

```
...     print(n, "is even")
```

```
... else:
```

```
...     print(n, "is odd")
```

```
5 is odd
```

Use the modulo operator (%) to calculate the **remainder**.
After dividing 10 by 3, the remainder is 1.

Check if a number is even.

Operators Summary

operator	operation
+	addition
-	subtraction
*	multiplication
/	float division
**	exponentiation
abs()	absolute value
//	integer division
%	remainder

Table 3.1: Python built-in numeric operations

Math library summary

Python	mathematics	English
<code>pi</code>	π	An approximation of π .
<code>e</code>	e	An approximation of e .
<code>sqrt(x)</code>	$\sqrt{x}$	The square root of x .
<code>sin(x)</code>	$\sin x$	The sine of x .
<code>cos(x)</code>	$\cos x$	The cosine of x .
<code>tan(x)</code>	$\tan x$	The tangent of x .
<code>asin(x)</code>	$\arcsin x$	The inverse of sine x .
<code>acos(x)</code>	$\arccos x$	The inverse of cosine x .
<code>atan(x)</code>	$\arctan x$	The inverse of tangent x .
<code>log(x)</code>	$\ln x$	The natural (base e) logarithm of x .
<code>log10(x)</code>	$\log_{10} x$	The common (base 10) logarithm of x .
<code>exp(x)</code>	e^x	The exponential of x .
<code>ceil(x)</code>	$\lceil x \rceil$	The smallest whole number $\geq x$.
<code>floor(x)</code>	$\lfloor x \rfloor$	The largest whole number $\leq x$.

Table 3.2: Some math library functions

Printing and Rounding

Unnecessary precision in the salary evolution

```
salary = 99000
incr = 3.5
years = 6
for y in range(1, years+1):
    salary = salary * (1 + incr/100)
    print("year", y, ":", salary)
```

```
year 1 : 102464.99999999999
year 2 : 106051.27499999998
year 3 : 109763.06962499997
year 4 : 113604.77706187496
year 5 : 117580.94425904058
year 6 : 121696.27730810699
```

We want to see only two decimals, up to an öre

Truncating and rounding

```
>>> float(4)
4.0
```

Integers to floating point, as expected

```
>>> int(4.0)
4
```

floating point to integer drops the fraction (towards 0). This is called truncating.

```
>>> int(4.1)
4
```

```
>>> int(-4.9)
-4
```

```
>>> round(4.9)
5
```

`round()` rounds as usual...

```
>>> round(3.5)
4
```

```
>>> round(4.50)
4
```

... ties go to the nearest even number.

Rounding x.5 up

```
def swedish(d):  
    return int(d + 0.5)
```

Inom så kallad *svensk avrundning* avrundar man **uppåt** om sista siffran man vill ha med följs av 5, 6, 7, 8 eller 9 och **nedåt** om sista siffran man vill ha med följs av 0, 1, 2, 3 eller 4.

Detta kommer från den svenska öresavrundningen i butiker

<https://sv.wikipedia.org/wiki/Avrundning>

Note: check if our function works right for negative numbers!

```
>>> swedish(0.5)
```

```
1
```

```
>>> swedish(1.5)
```

```
2
```

```
>>> swedish(2.5)
```

```
3
```

```
>>> swedish(3.5)
```

```
4
```

```
>>> swedish(4.5)
```

```
5
```

```
>>> swedish(5.5)
```

```
6
```

```
>>> swedish(5.49)
```

```
5
```

Rounding to a specific number of decimals

```
>>> round(4.51, 1)
```

```
4.5
```

```
>>> round(4.54256262, 1)
```

```
4.5
```

```
>>> round(4.54256262, 2)
```

```
4.54
```

```
>>> round(4.54256262, 4)
```

```
4.5426
```

```
>>> round(5425424.54256262, 4)
```

```
5425424.5426
```

```
>>> round(5425424.54256262, 0)
```

```
5425425.0
```

`round()` can take a second argument, which gives the number of decimals after the dot.

The result is a floating point, so we're still subject to rounding errors because the underlying representation is still binary.

(Not reliable when accumulating a value over multiple iterations!)

Unnecessary precision: solution

```
for y in range(1, years+1):  
    salary = salary * (1 + incr/100)  
    print("year", y, ":", round(salary,2))
```

```
# alternative, string formatting:  
print(f"year {y}: {salary:.2f}")
```

```
year 1 : 102465.0  
year 2 : 106051.27  
year 3 : 109763.07  
year 4 : 113604.78  
year 5 : 117580.94  
year 6 : 121696.28
```

Note that we round when we print out—not in the value of salary. Otherwise we will get a bigger **rounding error**. This is what engineers should do.

Unnecessary precision: (worse) solution

```
for y in range(1,years+1):  
    salary = round(salary * (1 + incr/100),2)  
    print("year",y,":",salary)
```

```
year 1 : 102465.0  
year 2 : 106051.27  
year 3 : 109763.06  
year 4 : 113604.77  
year 5 : 117580.94  
year 6 : 121696.27
```

Rounding while calculating → bigger rounding error.

(Accounting programs should use decimal representation of numbers, i.e. an integer number of *öre*, NOT a binary floating point representation of *kronor*.)

Rounding x.5 towards even

```
>>> round(0.5)
```

```
0
```

```
>>> round(1.5)
```

```
2
```

```
>>> round(2.5)
```

```
2
```

```
>>> round(3.5)
```

```
4
```

```
>>> round(4.5)
```

```
4
```

```
>>> round(5.5)
```

```
6
```

Clarification: Python 3 rounds to the nearest even number! (This method minimizes the expected error when summing numbers, even in the presence of negative numbers.)

Implementing Math: Examples

Sum of numbers and The Accumulator Pattern

```
def sum(n):  
    s = 0  
    for k in range(1,n+1):  
        s = s + k  
    return s
```

This computes:

$$1 + 2 + \dots + n$$

Programing pattern: **accumulator variable (accumulator) s**.

Example: $n = 6$

	$s = 0$
$k = 1:$	$s = 0 + 1 \rightarrow 1$
$k = 2:$	$s = 1 + 2 \rightarrow 3$
$k = 3:$	$s = 3 + 3 \rightarrow 6$
$k = 4:$	$s = 6 + 4 \rightarrow 10$
$k = 5:$	$s = 10 + 5 \rightarrow 15$
$k = 6:$	$s = 15 + 6 \rightarrow 21$
	return 21

Print a multiplication table

```
def multiTable(n):  
    for x in range(1, n+1):  
        row = ""  
        for y in range(1, n+1):  
            row = row + "\t" + str(x*y)  
        print(row)
```

*Programming pattern: generate pairs of values with **nested for-loops**.*

In addition: Do you see the accumulator variable in the above code?

```
>>> multiTable(10)  
  
1    2    3    4    5    6    7    8    9    10  
2    4    6    8    10   12   14   16   18   20  
3    6    9    12   15   18   21   24   27   30  
4    8    12   16   20   24   28   32   36   40  
5    10   15   20   25   30   35   40   45   50  
6    12   18   24   30   36   42   48   54   60  
7    14   21   28   35   42   49   56   63   70  
8    16   24   32   40   48   56   64   72   80  
9    18   27   36   45   54   63   72   81   90  
10   20   30   40   50   60   70   80   90   100
```

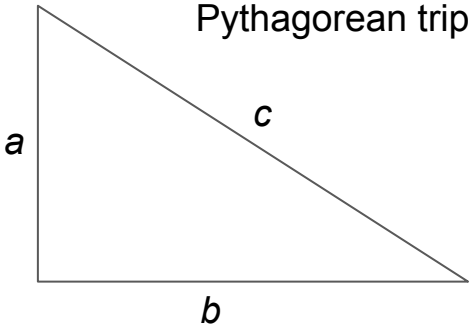
Generate Pythagorean triples

```
def pythagoreans(max):  
    for c in range(1, max):  
        for b in range(1, c):  
            for a in range(1, b):  
                if a*a + b*b == c*c:  
                    print(a,b,c,":",a*a,"+",b*b,"=",c*c)
```

Pythagorean theorem: in a right-angle triangle holds:

$$a^2 + b^2 = c^2$$

Pythagorean triples are integers a, b, c with this property.



```
>>> pythagoreans(32)  
3 4 5 : 9 + 16 = 25  
6 8 10 : 36 + 64 = 100  
5 12 13 : 25 + 144 = 169  
9 12 15 : 81 + 144 = 225  
8 15 17 : 64 + 225 = 289  
12 16 20 : 144 + 256 = 400  
15 20 25 : 225 + 400 = 625  
7 24 25 : 49 + 576 = 625  
10 24 26 : 100 + 576 = 676  
20 21 29 : 400 + 441 = 841  
18 24 30 : 324 + 576 = 900
```

Programming pattern: generate tuples (pairs, triples, etc) of values with nested for-loops.

Avoid repetitions of the same tuple by using the previous variable as upper limit i range().

Quadratic equations

A quadratic equation has the form $ax^2 + bx + c = 0$. Such an equation has two solutions for the value of x given by the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Zelle, p. 66

The expression under the square root, $b^2 - 4ac$, is called **discriminant**. It decides how many real solutions there are:

- If the discriminant is negative, there are no solutions
- If the discriminant is 0, there is one solution
- If the discriminant is positive, there are two different solutions

Quadratic equations: implementation

```
import math

def quadraticEq(a,b,c):
    discr = b**2 - 4*a*c
    if discr < 0:
        return []
    elif abs(discr) < 1e-10:
        return [-b/(2*a)]
    else:
        rdiscr = math.sqrt(discr)
        return [(-b - rdiscr)/(2*a),
                (-b + rdiscr)/(2*a)]
```

Note: The function always returns a list, which has 0, 1 or 2 elements depending on the discriminant. The elements are floating point numbers.

```
>>> quadraticEq(1,2,1)
[-1.0]
```

```
>>> quadraticEq(1,0,-9)
[-3.0, 3.0]
```

```
>>> quadraticEq(1,2,-1)
[-2.414213562373095,
 0.41421356237309515]
```

```
>>> quadraticEq(1,0,9)
[]
```

Approximation: the Newton-Raphson method for square root

```
def newton(r,n):  
    guess = r/2  
    for i in range(n):  
        guess = (guess + r/guess)/2  
        print(guess, "error:",  
              math.sqrt(r)-guess, sep='\t')  
    return guess
```

```
>>> newton(2,10)  
1.5          error: -0.08578643762690485  
1.4166666666666666 error: -0.002453104293571373  
1.4142156862745097 error: -2.123901414519125e-06  
1.4142135623746899 error: -1.5947243525715749e-12  
1.414213562373095 error: 2.220446049250313e-16  
1.414213562373095 error: 2.220446049250313e-16  
1.414213562373095 error: 2.220446049250313e-16  
1.414213562373095 error: 2.220446049250313e-16  
1.414213562373095 error: 2.220446049250313e-16  
1.414213562373095 error: 2.220446049250313e-16  
1.414213562373095 error: 2.220446049250313e-16  
1.414213562373095 error: 2.220446049250313e-16
```

The iterative formula to approach square root of r is:

$$x_0 = r/2$$

$$x_{n+1} = \frac{x_n + r/x_n}{2}$$

which is a special case of

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

which is a solution to the equation

$$f(x) = 0$$

with

$$f(x) = x^2 - r$$

The proof of the formula requires a bit of differential calculus and is left as an exercise for the interested.

Implementing Mathematics, Summary

- You can go a long way with expressions which combine standard functions.
- But sometimes it is necessary to write **algorithms**, often with loops
 - for example sums of numbers, factorial, etc.
- The behaviour of a numerical algorithm is not an obvious thing
 - be aware of the numerical characteristics of the algorithms you're using
 - precision, possible range, ...
 - even something as simple as rounding can be tricky!
- You are most likely to use a library (like numpy) with ready-made algorithms
 - be aware that in this case word-size integers and floats will be used, for efficiency.
 - be mindful of infinities, 'not a number', ...

Quiz (Numbers and loops)

Questions and answers in Mentimeter:

<https://www.mentimeter.com/app/presentation/albuw38hxqrgj-kx5u7j8ka1z115bz512/nmwcvcbe2jt67/edit>

References

From the book

- Chapter 3